

03 · Effort Estimation (UCP + COCOMO II)

Effort Estimation

Project	SDJ Editorial Portal — an editorial portal for the Science and Development Journal (SDJ), published by the College of Basic and Applied Sciences, University of Ghana
Document	03 — Effort estimation
Author	Roger Koranteng Obeng, student ID 22424140
Date	2026-08-12
Status	Authoritative. This document's MoSCoW cut (section 8) governs the scope of Plans 2–6.

1. Method

Per the design specification's estimation approach (section 14), **Use Case Points (UCP)** is the primary technique: the portal is defined by actor interactions with a stable use-case boundary — the situation UCP was designed for — and the unadjusted actor and use-case weights can be read directly off the functional requirements table (section 5.1) rather than guessed. **COCOMO II Early Design** serves as an independent cross-check from a size-and-cost-driver perspective, using different inputs entirely (source lines of code and process/product/personnel ratings rather than actor and transaction counts). Agreement — or a *diagnosable* disagreement — between two methods driven by different inputs is stronger evidence than precision from either one alone.

The estimate is computed before implementation. Its output determines the MoSCoW cut recorded in section 8: requirements that do not fit the available 48-hour window are demoted rather than rushed, and the demotion is recorded with its reason. Every intermediate figure below shows its arithmetic so a reader can recompute it independently rather than trust a stated total.

2. Actor inventory and unadjusted actor weight (UAW)

Actor weights follow Karner's classification: **Simple** (another system through a defined API) = 1, **Average** (another system through a protocol, or a human through a text interface) = 2, **Complex** (a human through a graphical interface) = 3.

Actor	Classification	Weight
Author	Complex — GUI	3
Reviewer	Complex — GUI	3
Editor	Complex — GUI	3
Editor-in-Chief	Complex — GUI	3
Administrator	Complex — GUI	3
Reader	Complex — GUI	3
Indexing service (OAI-PMH harvester)	Simple — defined API	1
UAW		19

Arithmetic: six GUI actors at weight 3 plus one API actor at weight 1 — $(6 \times 3) + (1 \times 1) = 18 + 1 = 19$.

3. Use-case inventory and unadjusted use-case weight (UUCW)

Use-case weights follow Karner's transaction-count classification: **Simple** (≤ 3 transactions) = 5, **Average** (4–7) = 10, **Complex** (> 7) = 15. The **FR** column traces each use case back to the functional requirement it implements in the design specification (section 5.1), so the inventory is derived rather than invented.

#	Use case	FR	Class	Wt
UC1	Register account	FR-01	Average	10
UC2	Authenticate and manage session	FR-02	Average	10
UC3	Manage roles	FR-03	Simple	5
UC4	Create submission	FR-04, FR-05	Complex	15
UC5	Process submission asynchronously	FR-06, FR-20	Complex	15
UC6	Screen submission	FR-07	Average	10
UC7	Recommend reviewers	FR-08	Complex	15
UC8	Invite and assign reviewers	FR-09	Average	10
UC9	Respond to review invitation	FR-10	Simple	5
UC10	Submit review	FR-11	Average	10
UC11	Record editorial decision	FR-12	Average	10
UC12	Submit revision	FR-13	Average	10
UC13	Track own submissions	FR-14	Simple	5
UC14	Compose and publish issue	FR-15	Complex	15
UC15	Browse archive	FR-16	Simple	5
UC16	Search papers	FR-17	Average	10
UC17	Download paper	FR-18	Simple	5
UC18	View audit trail	FR-19	Simple	5
UC19	Export citation	FR-21	Simple	5
UC20	Harvest metadata (OAI-PMH)	FR-22	Complex	15
UC21	Notify actors	FR-23	Average	10
UC22	View editorial analytics	FR-24	Average	10
UC23	Withdraw submission	FR-25	Simple	5
UC24	Configure journal policy	FR-26	Simple	5
UC25	Resolve persistent identifier	FR-27	Simple	5
UUCW				225

Arithmetic: summing the 25 weights in the Wt column —

$10+10+5+15+15+10+15+10+5+10+10+10+5+15+5+10+5+5+5+15+10+10+5+5+5 = 225$.

FR-28 (reviewer performance history, priority Could) has no corresponding use case here: it is a reporting feature over data already captured by UC10 and UC22 rather than an independent actor-facing transaction, so it does not add a separate UCP line; it is nonetheless carried in the technical debt register as a Could-have deferral consistent with its spec priority.

$$UUCP = UAW + UUCW = 19 + 225 = 244$$

4. Technical complexity factor (TCF)

	Technical factor	Wt	Rating	Product	Justification
T1	Distributed system	2	4	8.0	ECS, RDS, S3, Redis, CloudFront, Vercel
T2	Performance objectives	1	4	4.0	p95 targets in NFR-08, NFR-09
T3	End-user efficiency	1	3	3.0	Editorial queues optimised for throughput
T4	Complex internal processing	1	5	5.0	Hungarian assignment, MinHash LSH, hash chain
T5	Reusability	1	3	3.0	Hexagonal seams; multi-journal deferred
T6	Ease of installation	0.5	3	1.5	Terraform reproducible
T7	Ease of use	0.5	4	2.0	Public archive serves non-technical readers
T8	Portability	2	3	6.0	Containerised; adapters isolate vendors
T9	Ease of change	1	4	4.0	Hexagonal architecture is the point
T10	Concurrency	1	4	4.0	Async API plus worker pool
T11	Special security objectives	1	5	5.0	Double-blind, RBAC, tamper-evident audit
T12	Third-party direct access	1	4	4.0	OAI-PMH harvesting
T13	User training facilities	1	1	1.0	User manual only
	TFactor			50.5	

Arithmetic: $8.0+4.0+3.0+5.0+3.0+1.5+2.0+6.0+4.0+4.0+5.0+4.0+1.0 = 50.5$.

$$TCF = 0.6 + (0.01 \times 50.5) = 0.6 + 0.505 = 1.105$$

5. Environmental complexity factor (ECF)

	Environmental factor	Wt	Rating	Product	Justification
E1	Familiarity with process	1.5	4	6.0	Lifecycle taught this semester
E2	Application experience	0.5	3	1.5	Scholarly publishing newly studied
E3	Object-oriented experience	1	4	4.0	Strong
E4	Lead analyst capability	0.5	4	2.0	Sole analyst
E5	Motivation	1	5	5.0	Assessed final project
E6	Stable requirements	2	5	10.0	Self-defined and frozen at spec sign-off
E7	Part-time staff	-1	0	0.0	None
E8	Difficult programming language	-1	2	-2.0	Python and TypeScript are familiar
	EFactor			26.5	

Arithmetic: $6.0+1.5+4.0+2.0+5.0+10.0+0.0+(-2.0) = 26.5$.

$$ECF = 1.4 + (-0.03 \times 26.5) = 1.4 - 0.795 = 0.605$$

6. Use Case Points and full-system effort

$$\begin{aligned}
 \text{UCP} &= \text{UUCP} \times \text{TCF} \times \text{ECF} = 244 \times 1.105 \times 0.605 = 163.1 \\
 \text{Effort} &= \text{UCP} \times \text{PF} = 163.1 \times 20 = 3,262 \text{ person-hours} \\
 &\approx 21.5 \text{ person-months (at 152 h/month)} \\
 &\approx 1.8 \text{ person-years}
 \end{aligned}$$

Arithmetic: $244 \times 1.105 = 269.62$; $269.62 \times 0.605 = 163.12 \rightarrow 163.1$ UCP. $163.1 \times 20 = 3,262$ person-hours. $3,262 \div 152 = 21.46 \rightarrow 21.5$ person-months. $3,262 \div (152 \times 12) = 1.79 \rightarrow 1.8$ person-years.

Why PF = 20. Karner's productivity-factor rule counts how many of E1–E6 are rated below 3 and how many of E7–E8 are rated above 3, and sums the two counts. Here: E1–E6 are rated 4, 3, 4, 4, 5, 5 — none below 3, so that count is 0. E7–E8 are rated 0 and 2 — neither above 3, so that count is 0 as well. The total is 0, which sits within the documented ≤ 2 threshold for PF = 20 h/UCP (the standard alternative is PF = 28 at a total of exactly 3, with the project flagged for reconsideration above that). PF = 20 is therefore the correct rate for this factor profile, not a default assumption.

Must-have subset. Repeating the calculation restricted to UC1–UC18, with the Simple indexing-service actor excluded (its use case, UC20, is not in the Must-have set):

$$\begin{aligned}
 \text{UAW} &= 18 && (6 \text{ GUI actors} \times 3, \text{ indexing service excluded}) \\
 \text{UUCW} &= 170 && (\text{sum of UC1–UC18 weights: } 10+10+5+15+15+10+15+10+5+10+10+10+5+15+5+10+5+5) \\
 \text{UUCP} &= 18 + 170 = 188 \\
 \text{UCP} &= 188 \times 1.105 \times 0.605 = 125.7 \\
 \text{Effort} &= 125.7 \times 20 = 2,514 \text{ person-hours}
 \end{aligned}$$

Arithmetic: $188 \times 1.105 = 207.74$; $207.74 \times 0.605 = 125.68 \rightarrow 125.7$ UCP. $125.7 \times 20 = 2,514$ person-hours.

TCF and ECF are unchanged: they rate the *system* being built, not the subset of it in scope for a given milestone, so the same 1.105 and 0.605 apply to both calculations.

7. COCOMO II Early Design cross-check

$$\begin{aligned}
 \text{Size} &= 12 \text{ KSL0C} && (\text{backend } \sim 6, \text{ frontend } \sim 5, \text{ infrastructure } \sim 1) \\
 \text{Scale factors:} &\text{PREC } 3.72, \text{ FLEX } 2.03, \text{ RESL } 2.83, \text{ TEAM } 1.10, \text{ PMAT } 4.68 && \rightarrow \Sigma\text{SF} = 14.36 \\
 E &= 0.91 + 0.01 \times 14.36 = 1.0536 \\
 \text{Effort multipliers:} &\text{RCPX } 1.33, \text{ RUSE } 1.00, \text{ PDIF } 1.29, \text{ PERS } 0.63, \\
 &\text{PREX } 0.87, \text{ FCIL } 0.87, \text{ SCED } 1.43 && \rightarrow \prod\text{EM} = 1.1699 \\
 \text{PM} &= 2.94 \times 12^{1.0536} \times 1.1699 = 47.2 \text{ person-months} \approx 7,170 \text{ person-hours}
 \end{aligned}$$

Arithmetic: $\Sigma\text{SF} = 3.72+2.03+2.83+1.10+4.68 = 14.36$. $E = 0.91 + 0.1436 = 1.0536$. $\prod\text{EM} = 1.33 \times 1.00 \times 1.29 \times 0.63 \times 0.87 \times 0.87 \times 1.43 = 1.1699$. $12^{1.0536} = 13.71$. $\text{PM} = 2.94 \times 13.71 \times 1.1699 = 47.2$ person-months. $47.2 \times 152 \approx 7,170$ person-hours (the unrounded product of the full-precision intermediates, $47.155 \times 152 = 7,167.6$, rounds to the same figure at two significant figures; both paths land within a few hours of 7,170 and the discrepancy between them is rounding noise in the third significant figure, not a computational error).

Reconciliation. The two methods differ by roughly $2.2\times$ ($7,170 \div 3,262 = 2.20$). This is not a failure of either method; it is explained by what each one is built to measure. COCOMO II is calibrated on projects that carry formal verification, configuration management and management overhead that this project does not incur — a solo, ungoverned final project has none of the process weight the model's historical dataset assumes as baseline. Its $\text{SCED} = 1.43$ penalty for extreme schedule compression compounds multiplicatively with the

already-high RCPX (1.33, reliability and complexity) and PDIF (1.29, platform difficulty), so the compressed timeline is charged three times over in the effort-multiplier product. UCP, by contrast, counts only externally visible actor transactions; it is structurally blind to platform work — Terraform, CI/CD, the hash chain, the anonymisation pipeline's internals — of which this project has a great deal relative to its use-case count. Each method under-weights what the other over-weights. They do not converge on a single number, and are not expected to; what they do is **bound the answer from the same side**: both estimates exceed the 48-hour window by roughly two orders of magnitude, and **the full system is a roughly two-to-four person-year effort** by either reckoning (1.8 person-years from UCP on the full scope, up to $47.2 \div 12 \approx 3.9$ person-years from COCOMO II). That agreement — on the scale of the problem, not its exact magnitude — is what governs the scope decision in section 8.

8. Scope decision

48 hours is 1.5% of the lower (UCP, full-scope) estimate of 3,262 hours ($48 \div 3,262 = 0.0147$). Even against the Must-have subset alone (2,514 hours), 48 hours is 1.9% of the estimate. Under either reading, the available time is nearly two orders of magnitude short of what classical estimation predicts the full system requires. The estimate therefore governs scope in two ways: it forces a MoSCoW cut (below), and it forces an explicit reckoning with the fact that the build method itself — not just the schedule — has to absorb that gap (section 9).

8.1 MoSCoW cut — authoritative scope for Plans 2–6

Priority	Use cases	Decision
Must	UC1–UC18	Implemented to production quality
Should	UC19–UC23	Implemented only if Plans 2–5 complete early
Could	UC24, UC25	Deferred; entered in the technical debt register

This table is authoritative: Plans 2–6 treat it as the ranked scope contract. Should-have items are attempted only after every Must-have item is complete to production quality and time remains; Could-have items are not attempted within the 48-hour window under any circumstance and are recorded as deferred technical debt with a repayment plan rather than silently dropped.

9. Productivity reconciliation and AI-assisted development

The Must-have estimate of 2,514 person-hours assumes Karner's $PF = 20$ h/UCP, a rate calibrated on manual development. The realised effort for this project will be a small fraction of that figure, because this build is AI-assisted software development, conducted with Claude Code (Anthropic) pair-programming the implementation under the author's direction and review. That is a change of development **method**, not merely of pace — the same reason a productivity factor is described as a *calibration parameter* by the UCP literature rather than a physical constant: it encodes the rate at which a given method converts UCP into hours, and it must be re-derived whenever the method changes materially enough that the historical rate no longer describes it.

Four things follow from that:

1. **The realised PF is a local calibration, not a general claim.** It will be reported in section 10 once the build completes, computed as $\text{actual hours} \div \text{UCP of the delivered scope}$. It describes this developer, this tool, this domain and this 48-hour window. The sample size is one project, by one developer, and the figure does not generalise to other developers, other tools, or other problem domains — it is evidence about this build, not a claim about AI-assisted development in general.

2. **A lower PF does not mean the 2,514-hour estimate was wrong.** UCP still correctly sizes the *problem*; what changes is the rate at which the chosen method converts that size into elapsed hours. The estimate remains the correct basis for the MoSCoW cut in section 8, which was decided before the method's realised productivity was known.
3. **The gap between estimated and realised hours is not free capacity — it is capacity that was not spent on activities the classical estimate priced in.** Three of those are named here and cross-referenced forward to the technical debt register (not yet created at the time of writing; it is produced in a later implementation plan and each item below will carry a Debt → Cause → Impact → Priority → Proposed resolution entry per the design specification's technical debt policy, section 15):
 - **Test depth below what 2,514 hours would buy.** A fully-priced manual effort would include exhaustive edge-case testing, mutation testing and broader property-based coverage than a 48-hour window permits even with AI assistance. The domain and application layers meet the ≥85% line coverage gate (NFR-14) as a floor, not as evidence of exhaustive behavioural coverage.
 - **Documentation formality.** Architecture decision records, API reference documentation and onboarding material that a fully-staffed project would produce as a matter of course are reduced to what this estimation document, the design specification and inline code documentation provide.
 - **Security hardening.** Threat modelling, penetration testing and defence-in-depth beyond the NFR-01–NFR-06 baseline are not attempted within the window; NFR compliance is verified, but hardening beyond the specified baseline is not.
4. **Acknowledgement.** Consistent with the requirement to acknowledge all external resources and tools used in this work, AI assistance (Claude Code, Anthropic) is acknowledged here as a substantive contributor to implementation, and again in section 11 (References and acknowledgements), which records the specific tools and the boundary of their use: direction, review and final acceptance of all code and documentation rest with the author.

10. Assumptions, constraints and estimated-versus-actual

10.1 Assumptions

- A single developer builds the entire system.
- Requirements are frozen at spec sign-off (this design specification) and do not change materially during the build.
- System size is estimated at 12 KSLOC (backend ~6, frontend ~5, infrastructure ~1), used only as the COCOMO II sizing input.
- Managed AWS services (ECS, RDS, S3, CloudFront) are used rather than self-hosted equivalents, which is why the COCOMO II `FCIL` (facilities) multiplier is favourable at 0.87.
- Seed data is synthetic; no real SDJ submissions or reviewer data are used.

10.2 Constraints

- A 48-hour development window, which is the constraint the estimate is measured against in section 8.
- No registered domain is available, which shapes the CloudFront TLS strategy recorded in the design specification (section 16) rather than the estimate itself, but is recorded here as a project constraint.
- A solo developer, which is why the COCOMO II `TEAM` scale factor is rated at the Very High end (1.10, the lowest-penalty rating) — there is no cross-team coordination overhead to pay for — and why Karner's `E7` (part-time staff) is rated 0: there is no part-time staff to discount for.

10.3 Estimated versus actual

This section states the method now; the figures it produces are filled in at project close, once the build is complete and the commit history is final. Filling them in is a mechanical application of the method below, not a matter of judgement, so that the comparison is reproducible by a reader who was not present for the build.

Actual hours per phase, from commit history.

1. Extract every commit's timestamp and message: `git log --all --format='%H %ad %s' --date=iso-strict`.
2. Partition the ordered commits into working sessions: a gap of more than 45 minutes between two consecutive commit timestamps starts a new session. A session's duration is `last_commit_ts - first_commit_ts` within that session, floored at 15 minutes so that a session containing a single commit — which still represents real drafting, review and verification time not evidenced by a second commit — is not credited as zero.
3. Tag each commit to a phase (Plan 1–6, and within Plan 1 to a task) using its Conventional Commit type and message, which this project's implementation plans specify verbatim, task by task. Where a session spans commits from more than one phase, apportion that session's duration across the phases in proportion to the commit count each phase contributes to the session.
4. Sum apportioned session-hours by phase to give actual hours per phase, and sum all phases to give total actual hours.

Realised productivity factor. $PF_{actual} = \text{total actual hours} \div \text{UCP of the scope actually delivered at project close}$ — UCP taken from section 6's Must-have figure (125.7) if only Must-have use cases ship, or recomputed by the same method as section 3 over whichever Should-have use cases from section 8.1 are also delivered, if any are. This is the single figure section 9 describes as "a local calibration, not a general claim."

Variance percentage. $\text{Variance \%} = (\text{actual hours} - \text{estimated hours}) \div \text{estimated hours} \times 100$, computed against the Must-have UCP estimate (2,514 hours) as the primary comparison, since that is the estimate the MoSCoW cut in Section 8 was built against. A secondary variance figure against the COCOMO II estimate (7,170 hours) is reported alongside it, since section 7 established that the two methods bound the same answer from different sides and a large variance against one without checking the other would be misleading.

Hindsight re-rating of factors. At project close, each Technical (section 4) and Environmental (section 5) factor rating, and each COCOMO II scale factor and effort multiplier (section 7), is re-examined against what was actually observed during the build, and any rating that the build's evidence contradicts is flagged with what it should have been rated and why. Three ratings are already flagged here as pre-registered candidates for revision, precisely so that the hindsight analysis is not free to cherry-pick after the fact:

- **SCED (schedule compression, COCOMO II, rated 1.43).** This penalty models compression of a *manual* schedule. Whether the same penalty structure applies to an AI-assisted schedule compressed to the same degree is exactly the open question this project's realised PF speaks to, and is revisited directly.
- **T4 and T11 (complex internal processing and special security objectives, both rated 5).** These were rated from the design specification's description of the Hungarian assignment algorithm, MinHash LSH similarity screening and the hash-chained audit log before any of the three was implemented. Whether 5 (the maximum rating) was accurate is checked against how much of the build's actual effort those three subsystems consumed relative to the rest of the system.
- **PMAT (process maturity, COCOMO II, rated 4.68).** This scale factor ordinarily reflects an organisation's measured process maturity; here it was rated for a solo, ungoverned final project process, which is a judgement call revisited once the build's actual defect and rework rate — visible in the commit history as revert or fix-up commits following a feature commit — is known.

11. References and acknowledgements

- Karner, G. (1993). *Use Case Points* method for effort estimation, as codified in Cockburn, A. (2000), *Writing Effective Use Cases*, and Schneider & Winters (1998), *Applying Use Cases: A Practical Guide* — actor and use-case weighting rules (section 2, section 3) and the productivity-factor rule (section 6) follow this method.

- Boehm, B. et al. (2000). *Software Cost Estimation with COCOMO II* — Early Design model, scale factors and effort multipliers (section 7) follow this method.
- `docs/superpowers/specs/2026-08-12-ugjcs-journal-platform-design.md` — the design specification this estimate is derived from: section 5.1 supplies the functional-requirement inventory behind section 3's use cases; section 14 sets the estimation approach this document follows; section 15 sets the technical debt policy referenced in section 9; section 16 supplies the assumptions and constraints reproduced in section 10.
- **AI-assisted development.** This document, and the implementation plans and code it governs the scope of, were produced with Claude Code (Anthropic), an AI coding assistant, under the direction and review of the author. The assistant drafted prose and code from the author's instructions and the design specification; the author directed the work, reviewed every output, and accepts sole responsibility for its correctness and for the estimates and decisions recorded in this document. This acknowledgement satisfies the requirement to declare all external resources and tools used in this work, and is cross-referenced from section 9, where the productivity consequence of that method is analysed rather than merely disclosed.